
COMMON FOUNDRY

TECHNICAL WHITEPAPER / MAINNET LAUNCH EDITION



The operator-first route to open inference.

Compute-oriented mining. CPU-verifiable work. Direct service settlement. A technical foundation built for people who operate GPUs and the applications they can power.

SOURCE: OCTOBER 2

2026 | Both at noon CDT / 17:00 UTC | Planned launch schedule

Version 0.4 | September 24, 2026 | commonfoundry.ai

MINING: OCTOBER 3

Inside this edition

1. The case for Common Foundry	3
2. A product architecture with a clear next step	5
3. ForgeMatrix: compute with an exact acceptance rule	6
4. The proof: expensive work, independently checkable results	7
5. Integrity without trusting the miner	9
6. A launch designed around one shared start	10
7. CMFD: transparent distribution and durable incentives	11
8. Direct inference settlement: pay for the service you choose	12
9. Built for people who operate the network	14
10. Evidence that can be inspected	16
11. The next chapter: turn capability into demand	17
Appendix A. Protocol quick reference	18
Appendix B. Evidence and technical references	19

How to read this paper

Start with Sections 1-2 for the proposition. Sections 3-6 explain the computation, proof and shared launch. Sections 7-9 cover money, inference settlement and operator software. Sections 10-11 separate demonstrated results from the next product milestones. The appendices provide exact parameters and evidence pointers.

Launch status - September 24, 2026

Mainnet is not live yet. Source and launch packages are planned for October 2 at noon CDT / 17:00 UTC; mining is planned for October 3 at the same time. Final package and deployment checks remain in progress. Customer-paid inference is in development, not a feature of the initial mainnet launch.

1. The case for Common Foundry

1.1 AI compute should have more participants

The opportunity is to bring three things together: people who can operate compute, a network that can independently validate their mining work, and payment rules suited to buying useful services directly from those operators.

A capable GPU is only one part of a service business. Operators also need a way to agree terms, deliver work and collect payment. Customers need a clear scope, useful output and predictable settlement. Common Foundry is building toward those connections: an open operator base with shared payment rules, where delivering a service does not require the protocol to appoint one central seller or custodian.

Common Foundry's proposition is **inference first**. The long-term product is an open service economy in which customers choose providers and providers compete on what they deliver. The initial network establishes the operator base, settlement asset and technical foundation from which that economy can grow. Mining is the first participation path, not the final definition of the project.

The project takes an operator-first route. A miner can install software and contribute work before a complete inference marketplace exists. A developer can build wallets, miners, pool services or integrations against a concrete protocol. A community member can help another operator get online. Each contribution helps turn a protocol into infrastructure people can actually use.

The strategic thesis: recruit capable GPU operators through a functioning proof-of-work network, make their work independently checkable, then connect that operator ecosystem to customer demand through direct, metered settlement. This is a development strategy, not a claim that mining itself already fulfills AI requests.

1.2 The differentiators, and why they matter

Distinctive choice	What Common Foundry does	Why it matters
Compute-oriented mining	ForgeMatrix evaluates a fixed, block-bound, 384-layer matrix workload	Builds operational experience around matrix-capable GPUs, kernels, memory and power management
GPU production, CPU verification	Accelerators generate the work and proof; ordinary nodes verify on the CPU	Mining performance does not become authority over block validity; non-miners can validate
Transparent proof architecture	Structured sumchecks and a BaseFold commitment/opening argument connect every bank to the final output	Large matrix work is checked through a proof rather than replayed by every node
A shared launch boundary	A signed plan and a pinned future randomness-beacon round determine genesis	Early source access supports preparation without intentionally granting an advance-mining window
Direct provider settlement	Customer-signed cumulative payment states authorize payment to the selected service provider	Supports streamed service without a chain transaction for every output chunk
Explicit monetary rules	No premine or token sale; declining issuance, disclosed allocations, fee burning and a miner-only tail	Participants can calculate distribution and understand who receives newly issued CMFD
Multiple participation paths	Wallets, CPU nodes, pool search, solo proving and pool operation have different responsibilities	A newcomer does not need to run the entire GPU stack to join the network
Operationally grounded software	Encrypted wallets, signed artifacts, authenticated inputs and reorganization-aware pool accounting	Turns protocol ideas into deployable tools and identifiable operating responsibilities

The differentiation is the combination, not a claim that each primitive is unprecedented. Common Foundry specializes established cryptographic tools for its own workload and integrates them with a purpose-built Rust ledger and operator software. It is not a Bitcoin Core version or a renamed Bitcoin client. Familiar UTXO and cumulative-work concepts are combined with a different proof-of-work and proof-validation design. [P1-P4]

1.3 A reason to participate now

For miners, Common Foundry offers a compute-oriented network with solo and pool participation, visible work statistics and room for implementation competition. For developers, it offers a specialized proof workload and settlement primitives that can support new products. For infrastructure partners, it offers an identifiable path from operator onboarding to a focused service pilot.

For the wider community, the invitation is straightforward: join before launch, understand the rules, help test or build, and take part in the shared start. Participation does not require buying an allocation in a token sale. It does require treating hardware support, operating costs and service demand as things to measure rather than assume.

2. A product architecture with a clear next step

2.1 Mining and inference have different jobs

Mining secures the chain. Inference serves a customer. Keeping those roles separate is a deliberate product and consensus decision.

ForgeMatrix is a deterministic public workload. A validator needs one exact answer for a given block candidate. An inference service has different requirements: a customer selects a model, runtime, input, provider, price and delivery policy. Those choices should be able to evolve without turning every model deployment into a consensus change.

This separation also keeps block production independent of customer-job availability. The ledger can operate while the service market is being built, and providers can later deliver inference without also mining. Mining rewards and customer service payments are distinct revenue paths; neither is a promise of profitability.

2.2 Four layers, one direction

Layer	Responsibility	Launch relationship
Ledger and consensus	Ownership, signatures, issuance, difficulty, fees and channel spends	Foundation of the mainnet candidate
ForgeMatrix	Block-bound computation, work digest and transparent proof	Active proof-of-work/proof design
Operator software	Wallets, nodes, solo miners, pool search clients and pool accounting	Existing RC operation plus launch-candidate packaging
Inference service product	Provider discovery, quotes, execution, streaming and customer workflows	Next product layer; settlement primitives exist, end-to-end service remains in development

2.3 What is ready, and what comes next

The released RC environment has demonstrated wallet transfers, node synchronization, pool mining and full-sized proof production. The mainnet candidate adds a distinct launch identity, authenticated activation, encrypted prelaunch wallet preparation and updated operating protections. A running RC pool is evidence of operation, not evidence that mainnet has already launched.

The first inference pilot should be deliberately narrow: a defined model/runtime combination, a small provider cohort, a complete quote-to-settlement flow, and measurable customer experience. Expansion should follow completed jobs and repeat use. Provider count, time to first output, job completion rate and cost per delivered result will be more useful evidence than unsupported market-share claims.

The product sequence is therefore: **launch the network; improve operator deployment and efficiency; complete a focused paid-inference pilot; expand on demonstrated demand.** [P1, P5]

2.4 Why build a dedicated ledger?

A dedicated ledger makes ForgeMatrix verification, channel spending, fee burning and launch identity explicit consensus rules. They are enforced together rather than treated as claims supplied by an application operator. The tradeoff is real: Common Foundry must develop its own network participation, integrations and operating history. Using familiar primitives does not automatically inherit another chain's security or ecosystem.

3. ForgeMatrix: compute with an exact acceptance rule

3.1 Production geometry

The ProductionV4 relation uses 128 batch rows, dimension 4,096 and 384 sequential layers, organized into three banks of 128 layers. The public weight bank contains 6,442,450,944 raw weight bytes, exactly 6 GiB. The base table adds 524,288 bytes; the distributed model file also includes framing. This model bank is the public mining dataset, not a customer-selected trained inference model.

```
B = 128      D = 4,096      L = 384
p = 0x7f000001 = 2,130,706,433
F = GF(p)
EF = F[X] / (X^4 - 3)
```

Weight and base-table bytes from 0 through 250 are mapped through $x - 125$ into the field. Initial activation and each layer combine a challenge-derived coordinate mask with an elementwise cube:

```
A_0      = (Base + Mask_initial(challenge))^3
Z_1      = A_1 * W_1 + Mask_1(challenge)
A_(l + 1) = Z_l^3,          0 <= l < 384
```

The arithmetic is exact KoalaBear field arithmetic. Stored layout, index order, masks, reduction semantics and encoding are part of the protocol, not discretionary GPU settings. [P2-P3]

3.2 Why matrices, finite fields and cubic transitions?

Dense matrices create a substantial, regular workload with reusable data and a natural optimization target for GPU implementers. The production shape contains 824,633,720,832 logical matrix multiply-accumulate terms per complete evaluation and 201,850,880 cubic activation outputs including the initial activation.

Finite-field semantics provide exact cross-implementation answers and a direct algebraic language for proving those answers. A cubic transition adds a sequential nonlinear relation while remaining a low-degree polynomial that the proof system can check. The design is about making the entire computation bind together, not merely asking a miner to report that a matrix multiplication occurred.

The three-bank organization gives the prover a defined way to manage a large workload while preserving cross-bank constraints. Fixed weights can be authenticated and preprocessed; candidate-specific execution remains tied to the challenge.

3.3 An important precision about INT8

Earlier Common Foundry designs and descriptions emphasized INT8 x INT8 matrix multiplication. Compact byte-valued weights remain part of the data representation, but **the launch candidate's consensus relation is field-native, not simply a raw signed-INT8 GEMM**. Activations and arithmetic follow the field equations above. A kernel may exploit device-specific arithmetic, decomposition or specialized units only if its result matches those exact semantics.

This distinction matters for honest hardware claims. The protocol does not prove that a particular tensor core, GPU model or amount of physical VRAM was used. Nor does it establish permanent ASIC resistance. A faster implementation that produces the same valid relation and proof is legitimate competition; an implementation that changes the required result is not.

3.4 Work is bound to the block, not to a miner's story

The challenge commits to the full network identifier, parent block, transaction root, height, timestamp, target, algorithm version, proof version, proof-system identity, model-manifest identity and nonce. The transaction root commits the coinbase, including reward destinations.

The terminal activation consists of 524,288 canonical field values in row-major order. A domain-separated BLAKE3 digest binds those values to the challenge. A further work digest binds the algorithm, proof system, model, challenge and terminal digest. That work digest must meet the target independently derived by the node.

```
challenge = BLAKE3-DK(challenge_domain, canonical_block_inputs || nonce)
final      = BLAKE3-DK(final_domain,
                      challenge || field_count || canonical_final_values)
work       = BLAKE3-DK(work_domain,
                      versions || proof_system || model || challenge || final)
accept only if work <= expected_target and the complete proof verifies
```

These are explanatory abbreviations; the core binding specification gives the exact byte order and domains. Proof randomness is excluded from work identity. Re-encoding a proof is not another chance at the mining target. [P2]

4. The proof: expensive work, independently checkable results

4.1 A specialized transparent argument

Common Foundry uses a specialized transparent argument built around sumchecks and BaseFold polynomial commitments. It draws on the same broad family of algebraic and hash-based techniques associated with STARK-style systems, but the precise description is a **BaseFold-based transparent proof for the ForgeMatrix relation**. It is not a claim of private, zero-knowledge inference. The terminal activation is public, and this deployment does not make a general zero-knowledge privacy guarantee. [P3, R1]

The project-specific contribution is the relation, its block binding, claim routing, transcript, codec, GPU path and node integration. Common Foundry did not invent BLAKE3, Schnorr signatures, sumcheck or BaseFold. It composes and specializes them for a concrete mining workload.

Transparent verification avoids a secret proof-setup trapdoor. Public model and preprocessing artifacts still have exact identities and must be authenticated. Transparency does not mean that any arbitrary file, parser or implementation should be trusted.

4.2 What a valid proof establishes

Each bank has a fixed weight commitment and a dynamic execution commitment. Two relation repetitions per bank connect three properties:

1. **Matrix correctness:** the preactivation equals the committed matrix product plus the public mask.
2. **Layer continuity:** each layer consumes the preceding layer's activation, including boundaries between banks.
3. **Cubic correctness:** the next activation is the cube of the preactivation under the canonical field rules.

Opening claims bind these relations to the committed data. Twelve claims per bank are padded to sixteen in a specified order. The final bank is linked to the published final activation, whose BLAKE3 digest determines the candidate's work identity. This is the complete path from public model and block challenge to target-tested output, rather than a proof about an unrelated intermediate table. [P2-P3]

4.3 Transcript and opening parameters

Parameter	ProductionV4 value
Base field / extension	KoalaBear / degree 4, $X^4 - 3$
Challenger	KoalaBearDegree4Duplex; Poseidon2 width 16, digest 8
Fixed / dynamic columns	256 / 16
Row variables	23
BaseFold log blowup / queries	1 / 270
FRI rounds	23
Relation repetitions per bank	2
Batch / proof grinding	5 / 16 bits
Padded opening claims per bank	16

The matrix sumcheck has 19 variables and degree 3; the shift sumcheck has 7 variables and degree 2; the cubic sumcheck has 26 variables and degree 4. Opening reduction uses a 23-variable degree-2 sumcheck. Fiat-Shamir challenges follow the frozen domain and message order. These parameters describe the implementation; they are not a substitute for an end-to-end soundness analysis or an advertised security-bit estimate.

The transcript starts from the complete public-statement digest. Commitment order, relation order, claim routing, canonical field encoding and the pinned dependency revision all affect proof-system identity. Another implementation must reproduce them, not just recognize the same high-level equations.

4.4 Why CPU verification is central to the product

GPU search finds a candidate; replay and proving build its evidence. The producer performs a mandatory CPU self-check before submission. Receiving nodes independently validate the proof and normal ledger rules before accepting the block.

This division allows miners to improve accelerator code without making every user depend on that accelerator implementation. A wallet user or validating-node operator does not need a GPU. CPU verification still requires the appropriate authenticated artifacts, memory, storage and processing time; GPU-free is not resource-free.

4.5 Proof size: the actual deployed design

The full transparent proof is **12,025,320 bytes**, approximately 11.47 MiB. It contains a 16-byte header, 2,097,152 bytes of final-activation values, and three bank sections of 3,309,384 bytes each. The production profile allows a 13 MiB proof frame and a 16 MiB block frame.

The proof makes verification far cheaper than evaluating every matrix term, but it is not a sub-256-KiB object. The current design publishes the final table and hashes it directly; it does not deploy a compressed, succinct BLAKE3 argument for that table. Further proof compression is an optimization opportunity, not a launch claim.

At one continuously full 16 MiB block every 60 seconds, block payload alone would average about 2.24 Mbit/s and 22.5 GiB per day before database overhead. Real growth follows actual encoded blocks and block cadence. Bandwidth, storage and propagation must therefore remain first-class operator metrics.

5. Integrity without trusting the miner

5.1 Acceptance is a chain of checks

Attempted substitution	Relevant enforcement
Reuse work on another network or parent	Full network and parent binding in the challenge and transcript
Change transactions or reward destination	Transaction-root and coinbase commitment
Declare an easier target	Node-derived target plus target binding in the proof statement
Substitute weights or proof parameters	Authenticated model and proof-system identities
Supply an unrelated terminal activation	Final-bank opening claims, canonical terminal values and BLAKE3 digest linkage
Change a nonce while retaining the proof	Nonce-derived challenge, masks and transcript
Submit malformed or oversized proof bytes	Exact topology, bounded framing, canonical field parsing, no trailing bytes
Treat a pool share as an accepted block	Separate share, candidate-proof and node-admission stages

In September 24 internal checks, fresh complete proofs produced on an RTX 4090 and RTX 5090 were validated in separate verifier processes. Nine targeted tamper cases were rejected, including an alternate-network statement with recomputed digest claims. That last case matters: changing only the outer labels was not enough to make the old algebraic proof valid. [P6]

These are concrete regression results, not a declaration that cheating is mathematically impossible. Honest assurance names the enforced statement, assumptions and tested attacks. Implementation bugs, undiscovered algebraic shortcuts, majority-work attacks and network failures remain distinct concerns. There is no claimed external cryptographic audit in this edition.

5.2 Clear boundaries improve the business case

A proof of the ForgeMatrix relation is not proof that a customer received a correct AI answer. A valid pool share is not a matured mining reward. A signed release is not proof of bug-free software. Keeping those categories precise is useful to miners, customers and partners because it tells each participant what can be independently checked and what still requires operational judgment.

6. A launch designed around one shared start

6.1 Preparation and mining are separate milestones

Milestone	Scheduled time	Purpose
Public source and matching launch packages	October 2, 2026, noon CDT / 17:00 UTC	Inspect, build, download and prepare
Mainnet mining	October 3, 2026, noon CDT / 17:00 UTC	Begin work against the authenticated mainnet genesis

The preparation window is exactly 24 hours. Mainnet begins with a fresh chain; RC balances do not transfer. No premine or token sale supplies an earlier coin allocation.

Releasing software early is not, by itself, enough to prevent advance mining. Common Foundry therefore binds launch to a future, externally generated beacon result rather than relying only on a local clock or a promise not to start.

6.2 A plan-bound, beacon-derived genesis

The signed launch plan commits the economic rules, starting and easiest targets, reward destinations, proof/artifact identities, schedule and beacon policy. The candidate pins quicknet round **32,747,812**, scheduled for the announced mining start. It does not select whichever round happens to be newest when a node starts. [P7]

```
genesis = SHA256(  
  "CMFD/MAINNET/BEACON-GENESIS/V1\0"  
  || canonical_launch_plan_sha256[32]  
  || pinned_quicknet_chain_hash[32]  
  || pinned_round_big_endian_u64[8]  
  || verified_compressed_G1_signature[48])
```

The launch helper verifies the pinned BLS signature scheme and public key, and derives the result locally. It does not trust a relay's claimed randomness or substitute public key. Multiple relays provide transport alternatives for the same required round. If that round is delayed, the system waits for it instead of accepting operator-generated fallback entropy.

Node opening, mining work, replay and block admission use the authenticated runtime context. Prelaunch wallet preparation can create an encrypted wallet and show its receiving address without opening the live chain. After beacon verification, the user can unlock and connect. Source availability, wallet preparation and network activation are separate actions.

6.3 What fairness means here

This mechanism is designed to prevent useful advance work against a genesis that is not yet known, assuming the pinned beacon's threshold-security conditions hold and the launch gate is enforced. It does not promise identical Internet latency, identical hardware, or equal access to kernel optimization. It also introduces a launch-time dependency on the beacon's availability and future-round unpredictability. [R3]

The practical offer is a published preparation window, explicit parameters and a common authenticated start condition. That is stronger and more inspectable than an undisclosed launch time or an operator-controlled random seed.

7. CMFD: transparent distribution and durable incentives

7.1 A calculable issuance schedule

One CMFD equals 100,000,000 atomic units. The bootstrap spans 2,628,000 blocks, about five 365-day years at the 60-second target spacing. Real calendar duration depends on actual block production.

$$\begin{aligned}
 N &= 2,628,000 \\
 R_0 &= 50,000,000,000 \text{ atoms} \\
 R(h) &= \text{floor}(R_0 * (N - h + 1) / N), \quad 1 \leq h \leq N \\
 R(h) &= 500,000,000 \text{ atoms}, \quad h \geq N + 1
 \end{aligned}$$

Block one issues 500 CMFD. The bootstrap subsidy then declines linearly toward a very small final bootstrap reward. On block 2,628,001 the permanent 5 CMFD miner-only tail begins. The tail is a deliberate step up from the final bootstrap reward, not a hard supply cap or a schedule that simply stops declining at 5 CMFD. [P4]

7.2 Funding the network in public

During the bootstrap, 25% goes to stewardship, 5% to the community allocation, and the remainder to the miner. Integer-rounding remainder belongs to the miner. At the initial 500 CMFD subsidy, that means 350 CMFD to the miner, 125 to stewardship and 25 to the community allocation.

$$\begin{aligned}
 \text{steward} &= \text{floor}(R(h) * 25 / 100) \\
 \text{community} &= \text{floor}(R(h) * 5 / 100) \\
 \text{miner} &= R(h) - \text{steward} - \text{community}
 \end{aligned}$$

Recipient	Aggregate bootstrap CMFD
Miners	459,900,175.01312000
Stewardship	164,250,062.48688000
Community	32,850,012.48688000
Total	657,000,249.98688000

Both non-miner allocations end with the bootstrap. The 5 CMFD tail goes entirely to miners. The two allocation destinations are distinct addresses committed in the launch plan; at launch, **both wallets are controlled by the project owner**. The community label is a funding purpose, not a claim of decentralized voting, multisignature control or an independently governed treasury.

The commercial rationale is continuing engineering and ecosystem funding without an upfront token sale. Accountability still depends on how the funds are used and reported; the protocol fixes distribution, not spending quality. Miner coinbase outputs mature after 100 blocks. Stewardship/community outputs do not have that miner-maturity delay and can be consumed in a later block.

7.3 Burn transactions, preserve an ongoing miner budget

The mainnet candidate enforces a minimum 0.1 CMFD burn per non-coinbase transaction, including channel-close transactions. Fees are not added to miner coinbase rewards.

```
fee_burned = sum(input values) - sum(output values)
net_protocol_supply = gross_issuance - cumulative_burned_fees
```

At target spacing, the permanent tail issues 2,628,000 CMFD per 365-day year, about 0.4% of aggregate bootstrap issuance in its first year. Transaction activity removes coins through burning. Whether net supply increases or decreases depends on actual issuance and actual burned fees; the protocol does not guarantee net deflation or price appreciation.

The design separates two purposes: scheduled issuance supports network security, while transaction fees are a cost of using the ledger and reduce outstanding supply. Inference service payments are different: the provider is paid for the job, and only the transaction/channel-close fee is burned. **It would be incorrect to say that all inference revenue is burned.**

8. Direct inference settlement: pay for the service you choose

8.1 Why a payment channel?

Token-by-token inference produces many small delivery events. Requiring a new on-chain transaction for each event would attach chain latency, bandwidth and fees to the streaming loop. Common Foundry's cumulative payment-channel design moves authorization off-chain while retaining a defined on-chain settlement path.

A customer funds a channel for a specific provider and job. That provider earns the service payment directly; the payment is not divided among miners, stewardship or the community allocation. Providers can compete on models, pricing, latency and reliability independently of mining participation. [P5]

8.2 Exact terms, integer prices

A channel binds the network and job identifiers, customer/provider keys, model/runtime/input digests, deposit, close fee, base price, input/output token prices, token limits, output-chunk size and refund height.

```
provider_payment = base_price
+ ceil(input_tokens * input_price_per_1000 / 1000)
+ ceil(output_tokens * output_price_per_1000 / 1000)

deposit = provider_payment + customer_refund + close_fee_burn
```

Amounts are integer CMFD atoms. Exact conservation makes the provider payment, unused balance and burned fee independently computable. The signed terms prevent a party from silently substituting a different quoted model, runtime, input or price schedule.

8.3 Stream, acknowledge, settle

1. The provider signs a quote; the customer locks the agreed maximum deposit.
2. The customer signs an initial cumulative authorization covering the agreed initial charges and first output chunk.
3. The provider delivers a chunk and signs a receipt with token counts and a rolling output digest.
4. The customer checks delivery and authorizes the next cumulative state.
5. The provider can settle an authorized state without another customer interaction. At the refund height, the customer has the timeout-refund path.

After either terminal spend, the channel identifier is retired to prevent replay of an old state into a later channel. Cumulative states make progress explicit: an older authorized state pays the provider less, so the provider has an incentive to retain and settle the newest valid one.

Granularity bounds incremental exposure. The initial authorization includes the base/input charges and first output chunk; subsequent progress adds at most one configured chunk ahead. It is not a blanket guarantee that a customer's entire initial deposit or initial charge is risk-free.

8.4 Correct payment is not the same as correct inference

A receipt authenticates what the provider signed. It does not, by itself, prove that an arbitrary neural-network answer is correct, private or useful. Deterministic runtimes, repeated execution, spot checks, provider reputation or later verifiable-inference proofs are application choices for different workloads.

This boundary lets the ledger do what it can enforce precisely: authorize ownership transfers, honor channel terms and provide a timeout path. The product layer must supply discovery, job transport, actual model execution, delivery checks and customer experience.

Those settlement primitives are implemented in the source. A complete customer-paid inference marketplace is not part of initial mainnet. The next commercial milestone is a real end-to-end pilot, not a relabeling of mining as paid inference.

9. Built for people who operate the network

9.1 Three practical ways to join

Participation	What you run	What you contribute
Wallet or validating node	CPU-based validation and wallet/node software with authenticated artifacts	Ownership, transfers and independent rule enforcement; no GPU required
Pool search	A GPU search client connected to a pool	Candidate/share search; the pool handles winner proving and submission
Solo miner or pool operator	Search, full proving stack, node and operating infrastructure	Complete candidate production and, for pools, accounting and payouts

Pool search and solo proving have different setup footprints. Existing RC packages use roughly 6.4 GB of authenticated model data for the lighter role and roughly 61 GB of model/preprocessed inputs for the complete solo stack, plus working storage. Final launch-package instructions are authoritative for exact downloads and prerequisites. Windows workflows can include WSL2/Linux GPU-worker components; Windows support should not be mistaken for every proving component being a native Windows executable. [P1]

Hardware claims must also remain role-specific. Fresh September qualification exercised full proofs on RTX 4090 and RTX 5090 devices. Earlier qualification demonstrated a full proof path on an RTX 5070 Ti 16 GB. Pool-search measurements also covered an RTX 4070 Ti SUPER. These results do not certify every NVIDIA generation, every 8 GB card, every driver or every mixed-GPU rig.

9.2 An approachable wallet with explicit custody

The Rust-backed desktop wallet provides sending, receiving, balances, peer information and mining-related workflows. Encrypted live keys and backups use XChaCha20-Poly1305 with Argon2id-derived keys. Authenticated metadata binds the format, KDF parameters, network and wallet destination. Restore validates the recovered key and refuses to overwrite an existing wallet key. [P8]

Prelaunch preparation supports creating the mainnet wallet and an encrypted backup before activation. A public receiving address is not a private key. A backup password is not a network account reset token: losing the only usable key/backup credentials is a custody problem, not something a server administrator can reverse.

Signed manifests, checksums, pinned artifact identities and resumable input downloads help users install the intended software and data. These are distribution and integrity mechanisms; they do not replace software testing or guarantee uptime.

9.3 Pool accounting that treats reorganizations seriously

Pools offer frequent share feedback and a simpler client, but their accounting must distinguish an observed block from a canonical mature reward and a paid credit.

The launch-candidate source includes persistent payout holds when a previously credited or paid reward loses canonical mature backing. Affected payments and automatic retries pause. Exact signed transactions and reservations are retained to avoid accidentally issuing a conflicting replacement. Insufficient shared backing can hold the whole pool. Restarting does not clear the incident. [P9]

Reconciliation requires an explicit operator review against a fresh chain tip, the exact ledger generation and sufficient funds. The policy does not silently debit unrelated miners or recover losses through a hidden levy on future earnings. The dashboard can expose the hold without publishing private operator notes.

These protections are implemented and internally tested; this edition does not claim they have already been rolled out to the live RC service. Pool decentralization is also an adoption goal, not an accomplished fact: additional independently operated pools improve operator choice, while any pool operator remains responsible for its own policy and deployment.

9.4 Nodes, recovery and chain selection

The ledger uses UTXOs, 32-byte x-only secp256k1 public keys and 64-byte BIP340 Schnorr signatures. It checks canonical encodings, signatures, input availability/uniqueness, maturity, value conservation and channel rules. Consensus identifiers and signatures are network-bound and domain-separated. [P4, R2]

The target block interval is 60 seconds. Difficulty uses up to 180 effective header-work records, including startup history; it does not wait for 180 mined blocks before changing. Timestamp rules include a median of the preceding eleven timestamps. Cumulative work is checked with 512-bit arithmetic; a fully validated branch replaces the active chain only when its cumulative work is strictly greater.

The candidate separates its 5x-RC starting difficulty from the easier RC-level floor. The retarget uses the window's average target and a measured time span clamped between one-third and three times the expected span. That clamp is relative to the window average, not necessarily the immediately preceding target. Difficulty changes when blocks arrive; it cannot create missing hashrate or guarantee rapid recovery if block production stops.

Checksummed block logs, deterministic replay, controlled backups and authenticated startup state support recovery. Growing chain history still requires deliberate storage provisioning and retention policy. Pool holdings and already-signed transactions must be reconciled when recovering accounting data; restoring an old directory is not a license to pay the same credit again.

10. Evidence that can be inspected

10.1 Measured speed, with the measurement boundary intact

Hardware / date	Measured path	Recorded result
RTX 5090, earlier V4 baseline	Warm online proof construction only	6.093 seconds
RTX 5090, same baseline	Mandatory CPU self-verification	0.311 seconds
RTX 5090, same baseline	Peak proving allocation	8.080 GiB
RTX 5070 Ti 16 GB, earlier qualification	Complete three-process proof path	74.294 seconds; CPU verified
RTX 5090, September 21 qualification	Complete winning replay/proof path, first use / warm	56.409 / 18.557 seconds
Two isolated node instances, September 21	Normal CPU block admission	0.365-0.403 seconds in the recorded cases

The first four rows are historical baselines retained with their original scope. The September 21 complete path includes costs omitted from an online-prover-only number. These are not universal speed promises, average block times or an apples-to-apples hardware ranking. Search time, initialization, artifact authentication, proving, CPU checking and network propagation are distinct stages. [P6, P10]

The two September 21 full blocks survived node reopen, and CPU-only recovery without startup caches reconstructed the same height, tip and next target. Both node instances were exercised within one test process; this was not a separate-host network rehearsal.

On September 24, fresh full proofs from RTX 4090 and RTX 5090 devices passed separate-process cryptographic verification, including the BaseFold Merkle/query-fold checks and final low-degree checks. Other workloads remained active, so those smoke-run elapsed times are not presented as comparative benchmarks. The checks did not submit mainnet blocks. [P6]

10.2 Evidence of an operating release cycle

The RC community pool began operating September 6. A dated September 7 observation recorded 66 pool blocks and 7,836 accepted shares. Those numbers establish an early operational milestone, not current live counters or adoption scale.

The published miner.2 comparisons recorded console search rates of 20.285 FW/s on an RTX 4070 Ti SUPER, 27.59 FW/s on an RTX 5070 Ti, and 34.84 FW/s on an RTX 5090 under the documented conditions. The 5090 shared hardware with pool/proof activity. These are search rates, not full proofs per second. Accepted/rejected shares and stale work are necessary companions to console speed. [P1]

10.3 What this edition does not call complete

Mainnet publication, production service activation and the exact final-package smoke check are still ahead as of September 24. Native Windows/Linux application rebuilds matched in the recorded internal checks, but complete release reproducibility is not claimed while GPU-worker/package reconciliation remains open. The owner waived the extended six-hour GPU-dropout rehearsal; targeted proof/recovery checks do not substitute for that test.

Qualification is internal. Separate verifier processes and a separate verification path provide useful evidence, but are not an external audit or independent organizational reproduction. The selected release-approval policy uses one owner signer. The paper describes implemented controls and observed results without presenting them as an unconditional assurance of security.

11. The next chapter: turn capability into demand

Common Foundry's launch is a starting point for adoption, not the end of development. The near-term work should make it easier to become a reliable operator: install successfully, authenticate the right inputs, mine accepted work, understand power and latency, recover cleanly and upgrade predictably.

The next product step is paid inference. That means completing the parts a customer actually touches: finding a provider, receiving a quote, funding a job, getting useful output, authorizing progress and settling correctly. A pilot should demonstrate that entire flow before the project claims a working marketplace.

Next milestone	A meaningful success measure
Launch execution	Published source/packages, verified shared activation and working mainnet operations
Operator reach	Successful installs and accepted work across documented physical hardware configurations
Network resilience	Measured propagation, recovery, storage growth and multiple independently operated services
Proof efficiency	Lower complete-path latency, propagation cost and resource requirements under unchanged acceptance rules
Inference pilot	Completed customer-funded jobs, useful delivery, correct settlement and repeat usage
Service expansion	Additional providers/models supported by measured reliability and actual customer demand

There is no promised exchange listing, token price, investment return or guaranteed demand in this roadmap. The sales case is the product: a distinctive compute-oriented foundation, a public rule set, working operator software and a credible path into direct GPU services.

Your hardware. Your ideas. A place in the Foundry. Join the community, choose where you can contribute, and help build the inference layer people will want to use.

Website: <https://commonfoundry.ai/>

Community and launch guidance: <https://discord.gg/XGuutqWMWP>

Appendix A. Protocol quick reference

Parameter	Mainnet candidate / current V4 design
Source release / mining start	October 2 / October 3, 2026; both 17:00 UTC
Proof geometry	128 batch rows x 4,096 dimensions x 384 layers
Bank organization	3 banks x 128 layers
Raw weights / base table	6,442,450,944 / 524,288 bytes
Base field / extension degree	2,130,706,433 / 4
Public final activation	524,288 canonical fields; 2,097,152 bytes
Transparent proof	12,025,320 bytes
Proof frame / block frame caps	13 MiB / 16 MiB
Transaction frame / per-transaction inputs and outputs	64 KiB / 128 each
Block transaction / aggregate input / aggregate output caps	1,024 / 4,096 / 4,096
Block signature-check cap	2,048
Target spacing / retarget history	60 seconds / up to 180 records
Initial difficulty / easiest floor	5x RC / 1x RC
Miner coinbase maturity	100 blocks
CMFD precision	100,000,000 atoms per CMFD
Bootstrap / tail start height	2,628,000 blocks / 2,628,001
Initial subsidy / miner-only tail	500 CMFD / 5 CMFD per block
Bootstrap distribution	70% miner, 25% stewardship, 5% community; integer remainder to miner
Minimum non-coinbase fee burn	0.1 CMFD
Launch beacon	Pinned quicknet round 32,747,812
Release authorization	One owner signer; internal qualification

Retarget arithmetic

```
average_target = floor(sum(T_i) / Nw)
expected_span  = (Nw - 1) * 60
actual_span    = clamp(t_last - t_first,
                      expected_span / 3, 3 * expected_span)
next_target    = min(pow_limit,
                    max(1, floor(average_target * actual_span
                                / expected_span)))
work(T)        = floor(2^256 / (T + 1))
```

The formula applies when the selected history contains at least two records. With a single history record, its target is retained subject to the easiest-target cap. Targets use inclusive comparison. The normative implementation, not this explanatory table, defines all edge cases and encoding details.

Appendix B. Evidence and technical references

This edition was checked against the private launch-candidate source at commit

a8b23ec66a5a9f42bd2821408b6b9a886cffd392, with proof-qualification evidence for the reviewed fd319a9 source. The candidate source is scheduled to become public with the October 2 release. File references below identify the implementation evidence; they do not imply those private source files are already publicly accessible.

P1. Distributed RC software and performance. Common Foundry RC5 and miner.2 release notes; Windows/Linux setup, pool search, accepted-share observations and measurement conditions.

<https://github.com/JustAResearcher/CommonFoundry-Binaries/releases>

P2. Work and block binding. docs/consensus/production-v4-core-spec-v1.md, its canonical vector, and crates/cmfd-consensus/src/forgematrix_v4.rs. Exact challenge, final-activation, work and statement digest preimages; proof/artifact identity.

P3. Proof algebra and codec. docs/consensus/production-v4-proof-algebra-v1.md, production-v4-message-order-v1.json, and the candidate's prover/verifier implementations. The BaseFold dependency revision is 92b8eabaea9ab7306da5826caa700adabf7445ba.

P4. Monetary and ledger rules. crates/cmfd-consensus/src/economics.rs, difficulty.rs, network.rs, and chain validation. The mainnet release configuration fixes both target values and the two reward destinations.

P5. Service settlement. crates/cmfd-marketplace, consensus channel-lock validation, and docs/marketplace/payment-channels.md. Implementation state takes precedence over older planning language in historical notes.

P6. September qualification. Internal September 24 full-proof smoke records for SM89/RTX 4090 and SM120/RTX 5090, separate-process verification reports, and nine-case FRESH-PROOF-ADVERSARIAL-CHECKS.json. These records explicitly declare internal verification, not external audit, mainnet authorization or a comparative benchmark. Final-activation bytes agreed across the two devices; both full proofs were 12,025,320 bytes.

P7. Mainnet launch identity. docs/mainnet-readiness.md, docs/mainnet-target-binding-v2.md, the compiled mainnet release configuration, launch-plan parsing and launch-helper implementation. Historical notes can retain older pending states; the exact frozen source and signed release material govern the launch candidate.

P8. Wallet and distribution. docs/wallet-custody.md, wallet security implementation and release-integrity tooling. Argon2id/XChaCha20-Poly1305 wallet format, authenticated backup metadata, prelaunch preparation and no-overwrite restore.

P9. Pool reorganization protection. docs/pool-deep-reorg-policy.md and shared node/pool accounting implementation and tests. Source implementation is distinguished from live deployment.

P10. Performance boundaries and recovery. Earlier V4 qualification baselines and docs/mainnet-5x-live-qualification-20260921.md. Two complete proof/block paths, CPU node admission, reopened state and CPU-only cold recovery; the original scope limitations are retained.

R1. BaseFold. BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes. Primary research paper: <https://eprint.iacr.org/2023/1705>

R2. Schnorr and hashing primitives. BIP340 Schnorr signatures: <https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki> . BLAKE3 official implementation and specification links: <https://github.com/BLAKE3-team/BLAKE3>

R3. Launch beacon. drand protocol specification and security model: <https://docs.drand.love/docs/specification/> and <https://docs.drand.love/docs/security-model/> . Common Foundry pins its chain, key, round and verification scheme rather than accepting relay-selected identities.

The technical contribution is a specialized composition and an operating software stack. Public implementation, reproducible measurements, useful operator participation and completed service flows are how the project should continue earning confidence.